LAB 7 – Python testing

Testing applications has become a standard skill set required for any competent developer today. The Python community embraces testing, and even the Python standard library has good inbuilt tools to support testing. In the larger Python ecosystem, there are a lot of testing tools. Pytest stands out among them due to its ease of use and its ability to handle increasingly complex testing needs.

To get started, let's install pytest in our virtual env.

$ pip install pytest

Basic Pytest Usage

We will start with a simple test. Pytest expects our tests to be located in files whose names begin with test_ or end with _test.py.

Let's create a file called test_capitalize.py, and inside it we will write a function called capital_case which should take a string as its argument and should return a capitalized version of the string.

We will also write a test, test_capital_case to ensure that the function does what it says. We'll prefix our test function names with test_, since this is what pytest expects our test functions to be named.

```
# test_capitalize.py

def capital_case(x):
    return x.capitalize()

def test_capital_case():
    assert capital_case('semaphore') == 'Semaphore'
```

The immediately noticeable thing is that pytest uses a plain assert statement, which is much easier to remember and use compared to the numerous assertSomething functions found in unittest.

To run the test, execute the pytest command:

$ pytest

We should see that our first test passes.

We can notice that our function could lead to a bug. It does not check the type of the argument to ensure that it is a string. Therefore, if we passed in a number as the argument to the function, it would raise an exception.

We would like to handle this case in our function by raising a custom exception with a friendly error message to the user.

In our capital_case function, we should check that the argument passed is a string or a string subclass before calling the capitalize function. If it is not, we should raise a TypeError with a custom error message.

Let's try to capture this in our test:

```
# test_capitalize.py

import pytest

def test_capital_case():
    assert capital_case('semaphore') == 'Semaphore'

def test_raises_exception_on_non_string_arguments():
    with pytest.raises(TypeError):
        capital_case(9)
```

```
#capitalize.py

def capital_case(x):
    if not isinstance(x, str):
        raise TypeError('Please provide a string argument')
      return x.capitalize()
```

When we rerun our tests, they should be passing once again.

Using Pytest Fixtures

In the following sections, we will explore some more advanced pytest features. To do this, we will need a small project to work with.

We will be writing a wallet application that enables its users to add or spend money in the wallet. It will be modeled as a class with two instance methods: spend_cash and add_cash.

We'll get started by writing our tests first. Create a file called test_wallet.py in the working directory, and add the following contents:

```
# test_wallet.py


import pytest

from wallet import Wallet, InsufficientAmount

def test_default_initial_amount():
    wallet = Wallet()
    assert wallet.balance == 0


def test_setting_initial_amount():
    wallet = Wallet(100)
    assert wallet.balance == 100


def test_wallet_add_cash():
    wallet = Wallet(10)
    wallet.add_cash(90)
    assert wallet.balance == 100


def test_wallet_spend_cash():
    wallet = Wallet(20)
    wallet.spend_cash(10)
    assert wallet.balance == 10
```

```
def test_wallet_spend_cash_raises_exception_on_insufficient_amount():
    wallet = Wallet()
    with pytest.raises(InsufficientAmount):
        wallet.spend_cash(100)
```

First things first, we import the Wallet class and the InsufficientAmount exception that we expect to raise when the user tries to spend more cash than they have in their wallet.

When we initialize the Wallet class, we expect it to have a default balance of 0. However, when we initialize the class with a value, that value should be set as the wallet's initial balance.

Moving on to the methods we plan to implement, we test that the add_cash method correctly increments the balance with the added amount. On the other hand, we are also ensuring that the spend_cash method reduces the balance by the spent amount and that we can't spend more cash than we have in the wallet. If we try to do so, an InsufficientAmount exception should be raised.

Running the tests at this point should fail since we have not created our Wallet class yet. We'll proceed with creating it. Create a file called wallet.py, and we will add our Wallet implementation in it. The file should look as follows:

```
# wallet.py


class InsufficientAmount(Exception):
    pass


class Wallet(object):
    def __init__(self, initial_amount=0):
        self.balance = initial_amount


    def spend_cash(self, amount):
        if self.balance < amount:
            raise InsufficientAmount('Not enough available to spend {}'.format(amount))
        self.balance -= amount
```

```
def add_cash(self, amount):
    self.balance += amount
```

First of all, we define our custom exception, InsufficientAmount, which will be raised when we try to spend more money than we have in the wallet. The Wallet class then follows. The constructor accepts an initial amount, which defaults to 0 if not provided. The initial amount is then set as the balance.

In the spend_cash method, we first check that we have a sufficient balance. If the balance is lower than the amount we intend to spend, we raise the InsufficientAmount exception with a friendly error message.

The implementation of add_cash then follows, which simply adds the provided amount to the current wallet balance.

Once we have this in place, we can rerun our tests, and they should be passing.

$ pytest -q test_wallet.py

.....

5 passed in 0.01 seconds



Refactoring our Tests with Fixtures

You may have noticed some repetition in the way we initialized the class in each test. This is where pytest fixtures come in. They help us set up some helper code that should run before any tests are executed, and are perfect for setting-up resources that are needed by the tests.

Fixture functions are created by marking them with the @pytest.fixture decorator. Test functions that require fixtures should accept them as arguments. For example, for a test to receive a fixture called wallet, it should have an argument with the fixture name, i.e. wallet.

Let's see how this works in practice. We will refactor our previous tests to use test fixtures where appropriate.

test_wallet.py

import pytest

from wallet import Wallet, InsufficientAmount

@pytest.fixture

```
def empty_wallet():
    '''Returns a Wallet instance with a zero balance'''
    return Wallet()


@pytest.fixture
def wallet():
    '''Returns a Wallet instance with a balance of 20'''
    return Wallet(20)


def test_default_initial_amount(empty_wallet):
    assert empty_wallet.balance == 0


def test_setting_initial_amount(wallet):
    assert wallet.balance == 20


def test_wallet_add_cash(wallet):
    wallet.add_cash(80)
    assert wallet.balance == 100


def test_wallet_spend_cash(wallet):
    wallet.spend_cash(10)
    assert wallet.balance == 10


def test_wallet_spend_cash_raises_exception_on_insufficient_amount(empty_wallet):
    with pytest.raises(InsufficientAmount):
        empty_wallet.spend_cash(100)
```

In our refactored tests, we can see that we have reduced the amount of boilerplate code by making use of fixtures.

We define two fixture functions,wallet and empty_wallet, which will be responsible for initializing the Wallet class in tests where it is needed, with different values.

For the first test function, we make use of the empty_wallet fixture, which provided a wallet instance with a balance of 0 to the test.

The next three tests receive a wallet instance initialized with a balance of 20. Finally, the last test receives the empty_wallet fixture. The tests can then make use of the fixture as if it was created inside the test function, as in the tests we had before.

Rerun the tests to confirm that everything works.

Utilizing fixtures helps us de-duplicate our code. If you notice a case where a piece of code is used repeatedly in a number of tests, that might be a good candidate to use as a fixture.

Flask testing

Below is an API that takes a JSON input with integer values a and b e.g. {"a": 1, "b": 2}, adds them up and returns sum a + b in a JSON response e.g. {"sum": 3}.

```
# hello_add.py
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/add', methods=['POST'])
def add():
    data = request.get_json()
    return jsonify({'sum': data['a'] + data['b']})
```

Testing this API with pytest

We can test it with pytest

```
# test_hello_add.py
from hello_add import app
from flask import json

def test_add():
    response = app.test_client().post(
        '/add',
        data=json.dumps({'a': 1, 'b': 2}),
        content_type='application/json',
    )
```

```
    data = json.loads(response.get_data(as_text=True))

assert response.status_code == 200
assert data['sum'] == 3
```

Above you saw how to test a POST method in pytest. Below is how to test a get method

```
response2 = app.test_client().get('/api/users')
data = json.loads(response2.get_data(as_text=True))

data is an array of dictionary
```

```
assert "My name" in str(response2.data)
```

The Activity to be graded for this lab

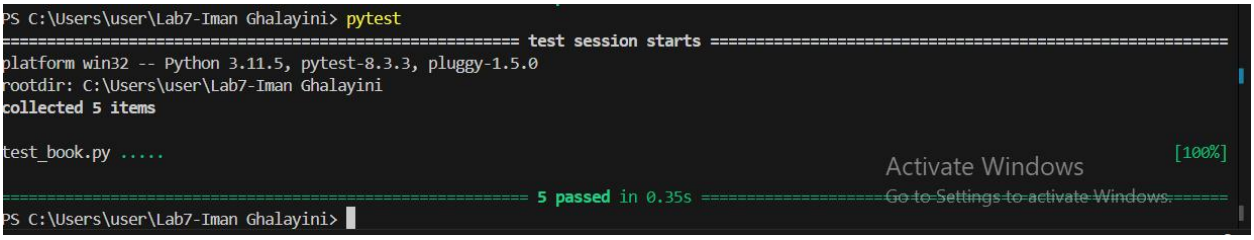
Using the project created in lab5, create the test_user_FirstName_LastName.py code that tests the following functions:

1. The data base is created with the specified users when first run the app
2. Insert_user()
3. Get_users()
4. Get_users_by_id()
5. Update_user()
6. Delete_user()
7. Api_get_user()
8. Api_add_user()
9. Api_update_user()
10. Api_delete_user()

For the api methods you should be testing the response status code and the data uploaded/downloaded

To be Uploaded to Moodle:

1. Zipped project containing all code for Lab7-FirstName-LastName
2. Snapshot showing the you tested Lab7-FirstName-LastName successfully same as bellow:


PS C:\Users\user\Lab7-Iman Ghalayini> pytest
test session starts
platform win32 -- Python 3.11.5, pytest-8.3.3, pluggy-1.5.0
rootdir: C:\Users\user\Lab7-Iman Ghalayini
collected 5 items
test_book.py .....
[100%]
5 passed in 0.35s
Activate Windows
Go to Settings to activate Windows.
PS C:\Users\user\Lab7-Iman Ghalayini>